

Übung 7

a) Bank

Fügen Sie Ihrer Bank-Klasse folgende Methoden hinzu:

- `public List<Kunde> getKundenMitLeeremKonto()`
Die Methode liefert eine Liste aller Kunden, die ein Konto mit negativem Kontostand haben. Wer mehrere überzogene Konten hat, soll nicht doppelt vorkommen.
- `public String getKundengeburtstage()`
liefert die Namen und Geburtstage aller Kunden der Bank, jeweils in einer Zeile. Doppelte Kunden sollen dabei aussortiert werden. Sortieren Sie die Liste nach Monat und Tag des Geburtstages (nicht nach dem Geburtsjahr!)
- `public long getAnzahlSenioren()`
liefert die Anzahl der Kunden, die jetzt mindestens 67 sind. Tipp: Die Klasse `Period` könnte helfen.
- `public void schenkungAnNeuerwachsene(Geldbetrag betrag)`
die Methode zahlt auf ein Konto jedes Kunden, der in diesem Jahr 18 wird, den betrag ein. (Hat ein gerade 18 Jahre alt werdender Kunde also zwei oder mehr Konten, wird nur eins davon beliebig ausgewählt.)

Benutzen Sie für diese Methoden Streams und Lambda-Ausdrücke, wo immer es geht, d.h. keine for-Schleifen (und eigentlich auch keine if-Anweisungen)!

Zum Ausprobieren können Sie die Klasse `Bankspielereien` aus Übung 4 verwenden.

b) Nullstellensuche

Sie kennen mathematische Funktionen, für die eine häufige Aufgabe lautet, eine Nullstelle zu finden, d.h. einen Wert x , für den $f(x) = 0$ ist. Sicherlich sind Sie dabei bisher eher analytisch vorgegangen, haben also die Funktionsgleichung umgeformt, um die Nullstelle zu errechnen. Der Computer ist aber im Probieren ganz gut, deshalb kann man auch so vorgehen, dass man sich Schritt für Schritt an die Lösung herantastet:

- Gehen Sie davon aus, dass eine Funktion f innerhalb eines Intervalles $[a, b]$ eine Nullstelle hat. Das ist auf jeden Fall richtig, wenn $f(a)$ und $f(b)$ verschiedene Vorzeichen haben, also eins kleiner als 0, das andere größer als 0 ist.
- Halbiert man das Intervall in zwei Hälften $[a, m]$ und $[m, b]$ (wobei m hier für die Mitte zwischen a und b steht) und betrachtet das Vorzeichen an den jeweiligen Teilintervallgrenzen, so kann man das Suchintervall auf denjenigen Teilbereich einschränken, in dem der Vorzeichenwechsel stattfindet.
- Diese Halbierung des Intervalles soll so lange fortgesetzt werden, bis die Größe des Intervalles kleiner ist als 0.01. Dann ist die Nullstelle genau genug berechnet.

Schreiben Sie eine Klasse mit einer statischen Methode zur Nullstellensuche: Sie soll eine beliebige¹ mathematische Funktion ($\mathbb{R} \rightarrow \mathbb{R}$) und die beiden ersten Intervallgrenzen a und b als Parameter bekommen. Hier bietet es sich an, wenn man die Funktion als Lambda-Ausdruck übergeben kann. Überlegen Sie sich gut, welcher Datentyp genau verwendet werden soll - Sie können ein eigenes Interface schreiben oder eins aus der Standardbibliothek verwenden.

Gehen Sie sinnvoll damit um, wenn die Funktion im gegebenen Intervall gar keine Nullstelle hat bzw. sie nicht gefunden werden kann.

Schreiben Sie einen JUnit-Test. Finden Sie als Testfälle mindestens die Nullstellen der Funktionen $f(x) = x^2 - 25$, $g(x) = e^{3x} - 7$ und $h(x) = \sin(x^2) - 0,5$. Die Intervallgrenzen können Sie selbst wählen. Probieren Sie auch, eine Nullstelle von $k(x) = x^2 + 1$ zu finden. Verwenden Sie Lambda-Ausdrücke.

- (1) Nun ja beliebig... Der Algorithmus klappt nur bei Funktionen, die im angegebenen Intervall stetig sind und eine Nullstelle mit Vorzeichenwechsel haben. Auch bei mehr als einer Nullstelle im Intervall kann es Schwierigkeiten geben.

Erzeugen Sie wieder mit dem Maven-assembly-Plugin ein (bzw. zwei) jar-Archiv(e) mit den Quelltexten und geben Sie es/beide ab.